Command Injection Finding Report

Enterprise DevSecOps Security Lab

Finding ID: APP-02

Command Injection in Flask /ping Endpoint

HIGH RISK

Document Type	Application Security Finding Report
Author	Jagriti Banerjee
Project	Enterprise DevSecOps Security Lab
Target Component	Vulnerable Flask application - /ping endpoint
Assessment Mode	Private VM lab / controlled local testing
Classification	Portfolio security assessment documentation

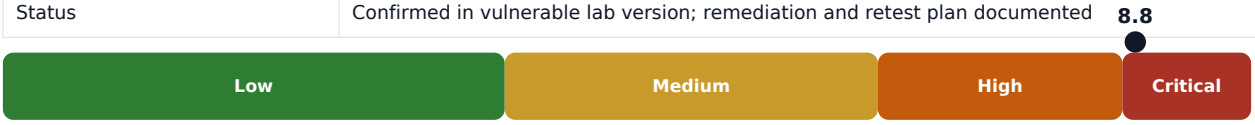
This report documents a command injection vulnerability discovered in the deliberately vulnerable Flask application used in the Enterprise DevSecOps lab. It includes risk scoring, exploit validation, root cause analysis, evidence, remediation guidance, OWASP/CWE mapping, and retest criteria.

1. Executive Summary

The assessment identified a command injection vulnerability in the Flask application route /ping. The route accepts a user-controlled host parameter and concatenates it into an operating system command that is executed with shell=True. A crafted payload appended additional shell commands after the intended ping command, proving that attacker-controlled input can be interpreted by the shell as executable operating system instructions.

The issue is intentionally present for private lab testing; however, the pattern represents a high-risk production weakness. In a real environment, command injection can allow an attacker to execute arbitrary commands in the application runtime context, enumerate the container, read accessible files, inspect environment variables, attempt lateral movement, or chain the issue with container/Kubernetes misconfigurations.

Field	Value
Finding ID	APP-02
Finding Name	Command Injection in Flask /ping Endpoint
Affected Route	/ping?host=<user supplied value>
Severity	High
CVSS v3.1 Style Score	8.8 - High (environment-adjusted lab score)
CWE Mapping	CWE-78 - Improper Neutralization of Special Elements used in an OS Command
OWASP Top 10 Mapping	A03:2021 - Injection
OWASP ASVS Mapping	Input validation, injection prevention, error handling, and secure design control areas
Status	Confirmed in vulnerable lab version; remediation and retest plan documented



Risk rating note: The score is calibrated for the private lab context where execution occurs inside a controlled VM and containerized application environment. If the same endpoint were internet-exposed, unauthenticated, connected to sensitive secrets, or deployed with privileged container permissions, the finding could reasonably be rated Critical.

2. Scope and Affected Asset

The finding applies to the intentionally vulnerable Flask application included in the Enterprise DevSecOps project. Testing was performed inside a controlled private VM lab environment using local requests against the application endpoint.

Asset	Details
Application	Python Flask vulnerable demo application
File	src/app/app.py
Function	ping()
HTTP Method	GET
Route	/ping
Parameter	host
Command Invoked	ping -c 1 <host>
Execution Method	subprocess.run(..., shell=True)
Test Type	Manual payload validation + source code review + Bandit SAST correlation

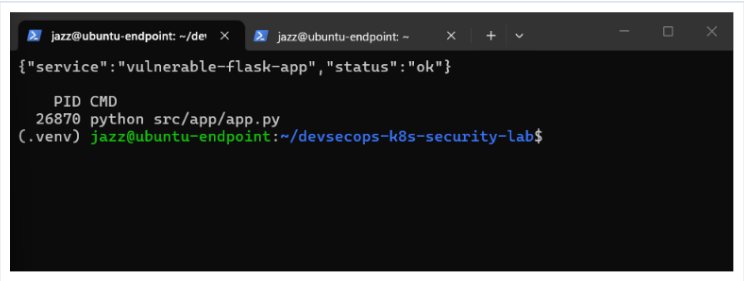


Figure 1 - Vulnerable Flask application running locally in the private lab.

3. Technical Vulnerability Details

The root cause is direct construction of an operating system command with untrusted input and execution through a shell. The application reads the host request parameter and inserts it into a command string. Because the command is executed with shell=True, shell metacharacters such as semicolon, ampersand, pipe, backticks, and command substitution can change the behavior of the executed command.

Vulnerable code pattern

```
@app.route("/ping")
def ping():
    host = request.args.get("host", "127.0.0.1")

    result = subprocess.run(
        f"ping -c 1 {host}",
        shell=True,
        capture_output=True,
        text=True
    )

    return f"<pre>{result.stdout}
{result.stderr}</pre>"
```

The key weakness is that user-controlled input is placed inside a shell-interpreted command. The shell receives the entire final string and treats injected separators as command syntax. This allows an attacker to execute another command after the intended ping command.

Case	Request Input	Effective Shell Interpretation	Result
Normal request	host=127.0.0.1	ping -c 1 127.0.0.1	Only the expected ping command executes.
Injected request	host=127.0.0.1; whoami	ping -c 1 127.0.0.1; whoami	The shell runs ping and then executes whoami.
Injected marker	host=127.0.0.1; echo COMMAND_INJECTION_WORKED	ping command followed by echo command	A tester-controlled marker appears in the response.

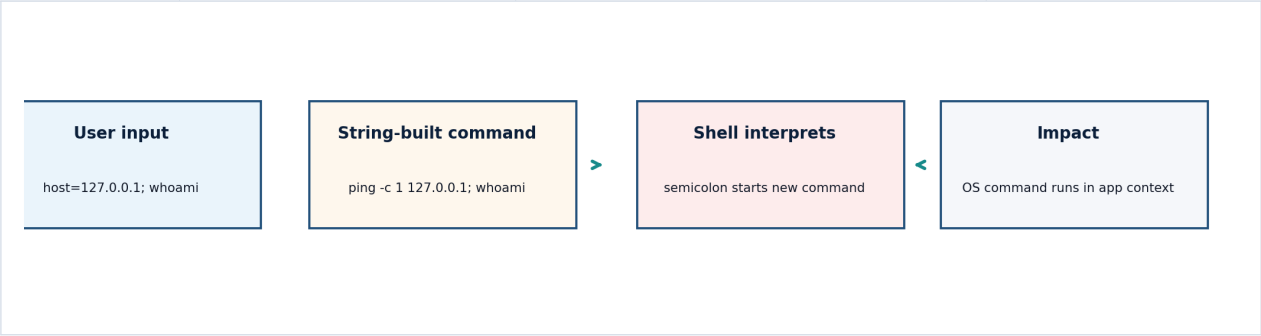


Figure 2 - Command injection logic flow for the vulnerable /ping endpoint.

4. Proof of Concept

The proof of concept was executed only against the local lab application. The goal was to validate whether the endpoint executes additional shell commands when command separators are inserted through the host parameter.

Baseline request

```
curl "http://localhost:5000/ping?host=127.0.0.1"
```

Expected baseline behavior: the application should return only the output of a ping request to localhost.

Injected request using identity command

```
curl "http://localhost:5000/ping?host=127.0.0.1%3Bwhoami"
```

Observed vulnerable behavior: the response included the output of the injected command, proving that input was executed by the operating system shell.

Injected request using a controlled marker

```
curl "http://localhost:5000/ping?host=127.0.0.1%3Becho%20COMMAND_INJECTION_WORKED"
```

Observed vulnerable behavior: the controlled marker was printed in the response, providing a safe non-destructive proof that the shell executed attacker-controlled input.

Figure 3 - Manual command injection proof showing injected command output from the /ping endpoint.

5. Root Cause Analysis

The application uses the operating system shell as an interpreter for a command string that includes untrusted request data. This creates a boundary failure: the program intends to pass a host value to ping, but the shell receives a full command line and interprets metacharacters as syntax.

Root Cause Area	Observation	Security Failure
Input handling	The host parameter is accepted as a raw string.	No allowlist validation for IP address or hostname format.
Command construction	The parameter is interpolated into an OS command string.	Data and shell syntax are mixed in the same execution string.
Execution method	The command is executed with shell=True.	Shell metacharacters can alter the command flow.
Output behavior	stdout and stderr are returned to the client.	The attacker receives direct feedback that helps confirm execution.
Testing coverage	The lab demonstrates the issue manually and with SAST.	Regression tests should be added after remediation.

Exploitability Factors

- The vulnerable parameter is reachable through a simple GET request.
- The lab route does not require authentication.
- The endpoint returns command output, allowing easy confirmation of execution.
- The vulnerable pattern uses shell=True, which gives special meaning to shell metacharacters.
- The proof of concept uses safe commands, but the same flaw could execute more sensitive commands if deployed insecurely.

6. Security Impact

In the private lab, command execution is limited to the local application runtime. In a production application, the same weakness could allow an attacker to interact with the server or container environment using the privileges of the application process. The impact depends on runtime permissions, mounted files, environment variables, network access, secrets exposure, and Kubernetes security posture.

Impact Area	Lab Observation	Potential Production Impact
Command execution	whoami and echo commands executed after ping.	Arbitrary command execution in application runtime context.
Confidentiality	Command output was returned in the HTTP response.	File disclosure, environment variable exposure, or secret discovery if permissions allow.
Integrity	The proof used non-destructive commands only.	Attackers may modify files, alter app behavior, or tamper with writable paths.
Availability	The proof used short commands only.	Attackers may run resource-intensive commands or disrupt the app process.
Cloud/Kubernetes risk	The lab later hardens runtime controls.	If combined with weak RBAC, hostPath mounts, or privileged pods, impact may escalate.

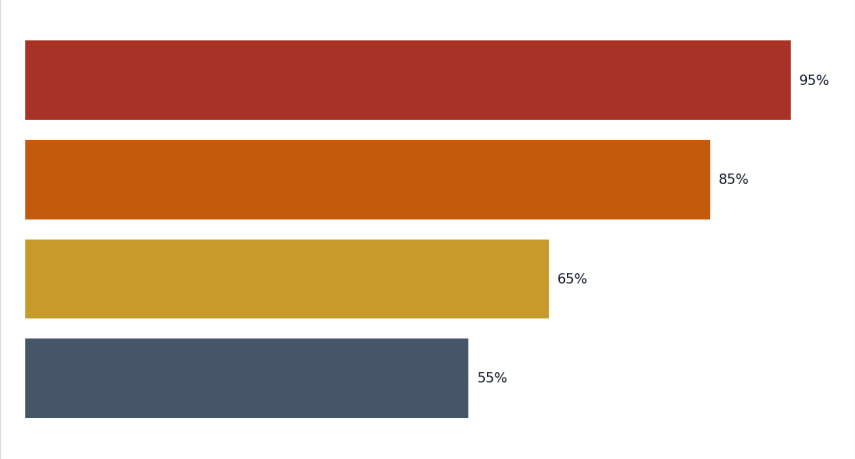


Figure 4 - Relative impact potential when command injection moves from lab to production.

7. Standards and Control Mapping

The finding maps to widely recognized injection weakness classifications. The core control requirement is to ensure user-controlled input cannot be interpreted as operating system command syntax.

Framework	Mapping	Reason
OWASP Top 10	A03:2021 - Injection	Command injection is an injection class where untrusted data changes command execution logic.
OWASP ASVS	Input validation, injection prevention, and secure design control areas	The application should validate input and avoid direct interpreter execution with untrusted data.
CWE	CWE-78	Improper neutralization of special elements used in an OS command.
OWASP Cheat Sheet	OS Command Injection Defense	Primary defenses include avoiding OS commands, validating input, using parameterized APIs, and avoiding shell interpretation.
Bandit	B602/B404 style subprocess findings	Static analysis flags shell=True and subprocess usage as risky patterns requiring review.

8. SAST and Evidence Correlation

The finding is supported by manual testing, source-code review, and Bandit SAST output. This correlation is useful for a professional AppSec workflow because it shows that the weakness is not only theoretically present in code but also reproducible at runtime.

Figure 5 - Bandit SAST evidence highlighting subprocess and shell execution risk.

Evidence correlation matrix

Evidence Source	What It Proves	Value to Remediation
Manual curl payload	Injected commands appear in HTTP response.	Confirms exploitability in the lab runtime.
Source code review	The route builds a shell command with f-string interpolation.	Identifies the exact root cause to fix.
Bandit SAST	Static tool flags shell=True/subprocess risk.	Can be used as a CI/CD guardrail after remediation.
OWASP/CWE mapping	Confirms recognized weakness category.	Supports risk prioritization and reporting consistency.

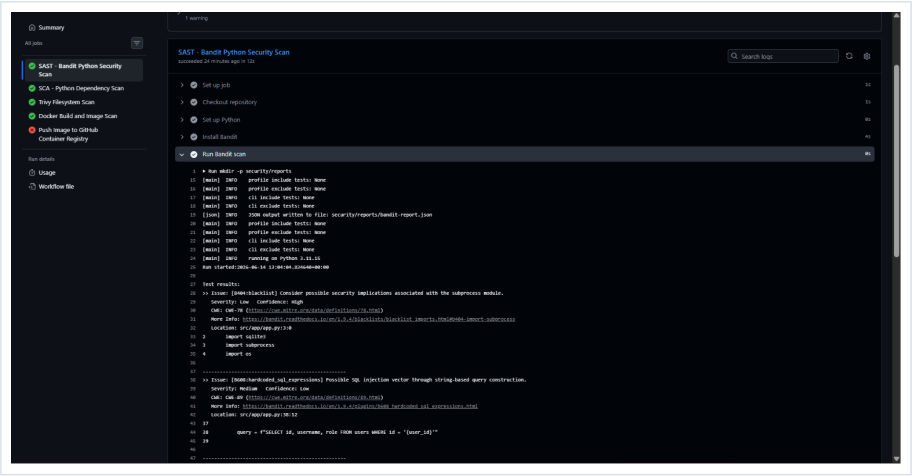


Figure 6 - CI/CD Bandit job evidence showing AppSec scanning integrated into the pipeline.

9. Remediation Guidance

The preferred remediation is to avoid operating system commands entirely where possible. If ping-like functionality is required for a lab or diagnostic feature, the implementation should not invoke a shell and should strictly validate input before passing it to any subprocess. Validation is necessary, but validation alone is not enough if shell interpretation remains enabled.

Recommended secure code pattern

```
import ipaddress
from flask import abort

@app.route("/ping")
def ping():
    raw_host = request.args.get("host", "127.0.0.1")

    try:
        safe_host = str(ipaddress.ip_address(raw_host))
    except ValueError:
        abort(400, description="Invalid host value")

    result = subprocess.run(
        ["ping", "-c", "1", safe_host],
        shell=False,
        capture_output=True,
        text=True,
        timeout=3
    )

    return {"status": "completed", "returncode": result.returncode}
```

Why this fixes the issue

- The command is passed as an argument list instead of a shell-interpreted string.
- shell=False prevents semicolons, pipes, and other shell metacharacters from starting new commands.
- The ipaddress check allows only valid IP address input and rejects injected command syntax.
- The timeout limits abuse of long-running commands.
- Returning a structured status reduces attacker feedback and avoids exposing raw command output.

Defense-in-depth recommendations

- Prefer a safe library or internal health check over invoking ping through the OS.
- Run the container as a non-root user with a read-only root filesystem where possible.
- Drop Linux capabilities and disable privilege escalation in Kubernetes manifests.
- Avoid mounting sensitive host paths or service account tokens into the application pod.
- Apply network egress restrictions so command execution cannot easily become data exfiltration.
- Keep Bandit and negative AppSec tests in CI/CD to prevent reintroduction.

Figure 7 - Recommended remediation and validation sequence.

10. Retest Plan and Acceptance Criteria

After the fix is applied, the endpoint should be retested with both valid and malicious input. The fix is not considered complete until normal functionality is preserved, injection payloads are rejected or treated as inert data, and automated security tooling no longer reports unsafe shell execution for the route.

Test ID	Test Case	Expected Result	Pass Criteria
CMDI-RT-01	GET /ping?host=127.0.0.1	Request completes normally.	No raw shell output required; response is structured and controlled.
CMDI-RT-02	GET /ping?host=127.0.0.1;whoami	Rejected as invalid host.	No whoami output, no command chaining.
CMDI-RT-03	GET /ping?host=127.0.0.1|id	Rejected as invalid host.	No piped command output.
CMDI-RT-04	GET /ping?host=$(whoami)	Rejected as invalid host.	No command substitution output.
CMDI-RT-05	Run Bandit against src/app	No shell=True finding for /ping route.	Bandit no longer flags the vulnerable pattern or finding is documented as safely mitigated.
CMDI-RT-06	Run application regression tests	Valid and invalid host tests pass.	Negative AppSec tests are included in CI/CD.

Recommended security unit tests

```
def test_ping_accepts_valid_ip(client):
    response = client.get("/ping?host=127.0.0.1")
    assert response.status_code == 200


def test_ping_rejects_command_separator(client):
    response = client.get("/ping?host=127.0.0.1;whoami")
    assert response.status_code in (400, 422)
    assert b"whoami" not in response.data


def test_ping_rejects_command_substitution(client):
    response = client.get("/ping?host=$(whoami)")
    assert response.status_code in (400, 422)
```

11. DevSecOps Integration Recommendations

The finding should be remediated in source code and reinforced across the delivery pipeline. The objective is to prevent any route from passing untrusted input to interpreters such as shells, SQL engines, template engines, or deserialization handlers without safe boundaries.

Control Layer	Recommended Control	Implementation Notes
Developer workflow	Secure coding checklist for subprocess use	Require shell=False, argument lists, validation, timeouts, and explicit approval for OS command execution.
Code review	Interpreter boundary review rule	Reviewers should flag any string-built command passed to subprocess, os.system, or shell wrappers.
SAST	Bandit job in GitHub Actions	Keep Bandit running on every push and pull request; review B602/B404-style subprocess findings.

Control Layer	Recommended Control	Implementation Notes
Testing	Negative AppSec tests	Add command injection payload tests for all routes accepting user-controlled command parameters.
Container security	Non-root + read-only runtime	Limit blast radius if application command execution occurs despite controls.
Kubernetes security	Pod Security, RBAC, NetworkPolicy	Reduce chaining opportunities from app compromise to cluster compromise.

12. Closure Checklist

Closure Item	Required Evidence
Vulnerable code removed	Code diff showing shell=True removed and host validation added.
Payload retested	Screenshots or test output proving command separators no longer execute.
SAST clean or accepted	Bandit report showing no unsafe shell execution for the route, or a documented safe exception.
Regression tests added	Unit/integration tests covering valid IP, semicolon injection, pipe injection, and command substitution.
Response hardened	No raw command output or internal errors returned to the client.
Runtime hardening retained	Kubernetes manifest keeps non-root execution, read-only filesystem, and dropped capabilities.
Pipeline updated	CI/CD job retains Bandit and negative AppSec tests on pull requests.

13. References

The following public references align the finding classification and remediation guidance:

- OWASP Top 10: A03:2021 - Injection, https://owasp.org/Top10/2021/A03_2021-Injection/
- OWASP OS Command Injection Defense Cheat Sheet, https://cheatsheetseries.owasp.org/cheatsheets/OS_Command_Injection_Defense_Cheat_Sheet.html
- OWASP ASVS Project, https://owasp.org/www-project-application-security-verification-standard/
- MITRE CWE-78, https://cwe.mitre.org/data/definitions/78.html
- Bandit security linter for Python, https://bandit.readthedocs.io/